Projet Final — DevOps : Suite

⚠️ **Attention — Lisez ceci avant de commencer**

Ce sujet est commun aux **deux groupes**. Certaines consignes varient selon votre groupe, veillez à bien appliquer **uniquement ce qui vous concerne** :

- **Groupe 1** → **Task API** : tout ce qui mentionne `task-api`, port `8085`, endpoints `/api/tasks`
- **Groupe 2** → **Product API** : tout ce qui mentionne `product-api`, port `8086`, endpoints `/api/products`

Appliquer les consignes de l'autre groupe **entraînera une perte de points**. En cas de doute, contactez-moi par mail à **chediallo99@gmail.com**

Introduction et Contexte

Ce projet final constitue une **continuité directe du premier projet** que vous avez réalisé (Mini Task Manager API ou Mini Product Inventory API). L'objectif est de consolider et d'évaluer votre maîtrise des concepts DevOps vus en cours, en particulier :

- Déploiement et gestion d'applications Spring Boot dans **Kubernetes (Minikube)**
- Monitoring et observabilité avec **Prometheus, Grafana et logs centralisés**
- Intégration continue et déploiement via **GitLab CI/CD**

Important :

- Si votre premier projet a été rendu correctement et dans les délais, vous pouvez **continuer directement sur votre travail existant**.
- Si vous avez rencontré des problèmes ou utilisé une **base de données différente de MySQL** (PostgreSQL, H2 ou in-memory), vous devez **corriger votre projet pour utiliser MySQL**.
  Tout écart **entraînera une perte de points**

Règles générales

1. Le projet doit être réalisé **en binôme** (2 étudiants maximum).

2. Tout **travail très similaire entre groupes** sera sanctionné pour plagiat.
3. Chaque étape doit être réalisée et testée (fonctionnelle ou avec justification en cas d'échec) et vérifiable via les **captures demandées et la vidéo de démonstration**.
4. Respect des technologies imposées :
   - **Minikube** pour Kubernetes
   - **MySQL** pour la base de données
   - **Prometheus/Grafana/Loki** pour monitoring et logs
   - **GitLab CI/CD** pour pipeline de build et déploiement Docker

---

Organisation des fichiers dans le projet

Tous les fichiers de configuration doivent se trouver **dans le code source** du projet principal :

- **Kubernetes** :
  - Fichiers `mysql.yaml` et `l3gl-devops.yaml` dans le répertoire `/k8s`
- **Observability / Monitoring** :
  - Fichier `prometheus.yml` et `docker-compose.yml` dans le répertoire `/observability`
- **GitLab CI/CD** :
  - Fichier `.gitlab-ci.yml` à la racine du projet

Note : Ces fichiers doivent être présents **uniquement dans le projet principal** :

- `task-api` pour le **premier groupe**
- `product-api` pour le **second groupe**

Architecture Projet

Description :

Le projet doit être déployé sur **deux VM (Linux Ubuntu) VirtualBox distinctes**, sauf si vous rencontrez des problèmes de stockage ou de performance, auquel cas vous pouvez utiliser un **cloud provider**.

Important : Tout doit être accessible et fonctionnel **le jour de la présentation** pour pouvoir tester et valider le projet.

1. **VM 1 – Application & Kubernetes**
   - Contient **Minikube** pour Kubernetes
   - Déploie l'application **Spring Boot** et **MySQL**
   - Expose l'application via un **Ingress** et un **reverse proxy Nginx** pour accéder depuis l'extérieur
2. **VM 2 – Monitoring**
   - Contient **Docker Compose** pour monter :
     - **Prometheus** (scraping des métriques Spring Boot)
     - **Grafana** (visualisation des métriques et dashboards)
     - **Loki** (centralisation des logs)
   - Prometheus scrappe l'application Spring Boot via l'**IP publique de la VM 1** + chemin `/l3gl/actuator/prometheus` (le reverse proxy Nginx)

Schéma Architecture

```
+----------------+                                    +----
------------+
| VM 1           |                                    | VM
2           |
| (Kubernetes)   |                                    | (Mo
nitoring)   |
|                |                                    |
|
| +------------+ |  <-- scrape /actuator/prometheus-- | +--
----------+ |
| | Spring     | |                                    | | P
rometheus | |
| | Boot App   | |  -- logs push (Loki) ----------->  | +--
----------+ |
| +------------+ |                                    | +--
----------+ |
| +------------+ |                                    | | G
```

```
rafana    | |
| | MySQL      | |                                    | +--
----------+ |
| +------------+ |                                    | +--
----------+ |
| +------------+ |                                    | | L
oki       | |
| | Nginx RP   | |                                    | +--
----------+ |
| +------------+ |                                    |
|
+----------------+                                    +----
------------+

Accès externe VM1 : http://<IP-VM1>/l3gl/*
```

Explications :

- **VM 1** héberge l'application et la base de données dans Kubernetes. L'accès externe est géré par **Nginx reverse proxy**.
- **VM 2** est dédiée au monitoring. **Prometheus** récupère les métriques de l'application via l'IP de la VM 1 et le chemin exposé `http://<ADRESSE-IP-VM1>/l3gl/actuator/prometheus`.
- Les logs de l'application Spring Boot sont centralisés dans **Loki**, puis visualisés dans **Grafana**.

Livrables finaux

Chaque groupe devra remettre :

1. **Liens des dépôts à jour** :
   - **GitHub** : code source complet avec les fichiers `/k8s` et `/observability`
   - **GitLab** : même code source avec le fichier `.gitlab-ci.yml`, le pipeline doit avoir tourné au moins une fois avec succès. Tout dépôt privé ou pipeline jamais exécuté **entraînera automatiquement la perte des points associés à cette partie**.
2. **Lien Notion public** détaillé incluant :
   - Explications de chaque partie (Kubernetes, Monitoring, CI/CD)

- Captures de fichiers de configuration, dashboards, pods, services, ingress, logs, etc.
- Captures Docker, Docker Hub et conteneurs en fonctionnement
- Contenu des fichiers dans `/k8s` et `/observability` et `.gitlab-ci.yml`

3. **Vidéo de démonstration** d'une durée comprise entre 7 et 15 minutes, montrant :
   - ~3 min → Kubernetes : pods, services, ingress et reverse proxy fonctionnels
   - ~2 min → Tests de l'application déployée dans la VM (requêtes API fonctionnelles, accès interne et externe)
   - ~2 min → Monitoring et logs visibles dans Grafana (dashboards GET/POST + logs Loki)
   - ~2 min → Pipeline GitLab CI/CD (jobs qui s'exécutent, image poussée sur Docker Hub)
   - ~1 min → Application fonctionnelle ou capture des erreurs éventuelles
4. **Envoi par email** en respectant les consignes suivantes :
   - Préciser le groupe auquel vous appartenez dans le contenu du mail
   - L'objet du mail doit être exactement sous la forme :

`Projet Final DevOps – <Membre du groupe> <Numéro du groupe>`

Important : Tout dépôt privé ou livrable incomplet ne sera pas corrigé.

Présentation

- Chaque groupe devra faire une **présentation de 10 minutes maximum**.
- Pendant cette présentation, vous devrez :
  - Expliquer ce que vous avez réalisé
  - Décrire les difficultés rencontrées
  - Expliquer comment vous les avez résolues
- Des **questions seront posées à chaque membre du groupe**.
- **Les notes de la présentation sont individuelles**, même si le travail est en binôme.

Pénalités éventuelles

- Projet rendu **hors délai** : réduction de points selon la gravité.
- Utilisation d'une **base de données autre que MySQL** : entraînera une perte de points importante
- Travail **copié ou très similaire à un autre groupe** : sanction maximale (zéro pour la partie concernée ou le projet complet).
- Livrable incomplet, fichiers manquants dans le code ou vidéo non conforme : points retirés pour les parties manquantes.

Partie 1️⃣ : Kubernetes – Déploiement de l'application Spring Boot avec MySQL

Objectif

Évaluer votre capacité à déployer votre application dans un cluster **Minikube**, avec MySQL, en configurant correctement les ressources Kubernetes et en rendant l'application accessible depuis l'extérieur via un reverse proxy Nginx.

Consignes de déploiement

1. **Namespace** :
   - Toutes les ressources doivent être créées dans le namespace : `devops-l3gl`
2. **Base de données MySQL** :
   - Déployer MySQL dans le même namespace.
   - Créez un **Deployment** et un **Service ClusterIP** (`port 3306`).
   - Le mot de passe root doit être défini via **ConfigMap ou variable d'environnement ou Secret (recommandé)** et monté dans le pod.
3. **Application Spring Boot** :
   - Créez une **ConfigMap** pour stocker les variables de configuration et montez-la dans le pod.
   - Déployez votre application via un **Deployment**.
   - Créez un **Service ClusterIP** pour exposer votre application **uniquement dans le cluster** :

Product Api → `port 8086`
Task Api → `port 8085`

4. **Actuator pour probes** :
   - Intégrer **Spring Boot Actuator**.
   - Exposer les endpoints `/actuator/health/readiness` et `/actuator/health/liveness`.
   - Configurer les **ReadinessProbe** et **LivenessProbe** sur ces endpoints.
5. **Ingress et Reverse Proxy** :
   - Créez un **Ingress HTTP** pour router vers le service de l'application (`/`).
   - Configurez un **reverse proxy Nginx** sur la VM pour exposer l'application publiquement :

```
http://<ip-vm>/l3gl/*
```

   - Le reverse proxy doit rediriger `/l3gl/*` vers l'Ingress Minikube `/`.
6. **Limites de ressources** :
   - Vous devez définir les **requests** (ressources minimales garanties) et les **limits** (ressources maximales autorisées) pour le conteneur de votre application Spring Boot. Ces valeurs permettent à Kubernetes de gérer correctement l'allocation des ressources dans le cluster.
   - Limites et requests :

```
CPU request: 200m       | CPU limit: 400m
Memory request: 250Mi   | Memory limit: 500Mi
```

7. **Fichiers YAML** :

- Les ressources doivent être réparties dans **deux fichiers distincts** :

Fichier	Contenu
`mysql.yaml`	ConfigMap ou Secret MySQL, Deployment MySQL, Service MySQL
`l3gl-devops.yaml`	ConfigMap app, Deployment Spring Boot, Service app, Ingress

- Chaque ressource séparée par `---` dans chaque fichier

- Les deux fichiers doivent être placés dans le répertoire `/k8s`

Livrables attendus Kubernetes

1. **Fichiers** `mysql.yaml` et `l3gl-devops.yaml`
2. **Captures d'écran** :
   - Namespace : `kubectl get namespaces`
   - Pods : `kubectl get pods -n devops-l3gl`
   - Deployment : `kubectl get deployment -n devops-l3gl`
   - Service : `kubectl get service -n devops-l3gl`
   - Ingress : `kubectl get ingress -n devops-l3gl`
   - Test en local et externe (via Nginx `/l3gl/*`)
   - Logs du pod : `kubectl logs <pod> -n devops-l3gl`
   - Capture des fichiers `mysql.yaml` et `l3gl-devops.yaml`

Critères d'évaluation Kubernetes (10 points)

Critère	Points	Notes
Namespace correct (`devops-l3gl`)	0.25	Toutes les ressources doivent être dans ce namespace
MySQL Deployment	0.5	Pod MySQL correctement configuré et démarré
MySQL Service ClusterIP	0.5	Service MySQL fonctionnel sur port 3306
ConfigMap utilisé et monté correctement	1	Variables montées dans le pod de l'application Spring Boot
Deployment application configuré correctement	2.5	Pods avec probes actuator et ressources définies
Service ClusterIP application opérationnel	1	Service exposant le pod sur port → **8085 (Task API) / 8086 (Product API)**
Ingress HTTP opérationnel	1	Routage correct vers le service depuis l'Ingress
Reverse proxy Nginx configuré et application accessible via `/l3gl/*`	1.75	Accès public fonctionnel

Critère	Points	Notes
ReadinessProbe et LivenessProbe via actuator configurées	0.5	Endpoints `/actuator/health/readiness` et `/actuator/health/liveness`
Limites et requests CPU/Memory correctes	1	Requests/limits respectés (CPU: 200m/400m, Memory: 250Mi/500Mi)

Total : 10 points

Partie 2️⃣ : Monitoring et Observabilité – Prometheus, Grafana et Logs

Objectif

Évaluer votre capacité à configurer un système de monitoring et d'observabilité pour l'application. Collecte des métriques via Prometheus, visualisation via Grafana et logs centralisés via Loki, montés via **Docker sur une VM séparée**.

Consignes Monitoring

1. **Application Spring Boot** :
   - Ajouter la dépendance Prometheus dans le `pom.xml` :

```
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

   - Activer les endpoints dans `application.properties` :

```
management.endpoints.web.exposure.include=health,info,prometheus,metrics
management.endpoint.prometheus.enabled=true
```

   - Endpoint `/actuator/prometheus` pour le scraping.
   - Logs centralisés vers Loki.
   - Ajouter la dépendance Loki dans le `pom.xml`

```
<dependency>
    <groupId>com.github.loki4j</groupId>
    <artifactId>loki-logback-appender</artifactId>
    <version>1.4.1</version>
</dependency>
```

- Créer un fichier `logback-spring.xml` dans `src/main/resources` :

```
<appender name="LOKI" class="com.github.loki4j.logback.L
oki4jAppender">
    <http>
        <url>http://<ADRESSE-IP-VM2>:3100/loki/api/v1/pu
sh</url>
    </http>
    <format>
        <label>
            <pattern>app=${spring.application.name}</pat
tern>
        </label>
        <message>
            <pattern>%-5level [%thread] %logger{36} - %m
sg%n</pattern>
        </message>
    </format>
</appender>
```

- Remplacer `<ADRESSE-IP-VM2>` par l'IP de votre VM Monitoring.

⚠️ Assurez-vous que le logger Loki est bien activé dans la configuration Logback, sinon aucun log ne sera visible dans Grafana.

1. **Prometheus** :
   - Scraper l'application sur `/actuator/prometheus` — **port 8085 (Task API) / port 8086 (Product API)**
   - Intervalle de scraping : **10 secondes**.
2. **Grafana** :
   - Datasources : Prometheus (métriques) et Loki (logs).

- Créer **deux dashboards** :
  - **Groupe 1 → Task API** :
    1. Nombre d'appels **GET /api/tasks**
    2. Nombre d'appels **POST /api/tasks**
  - **Groupe 2 → Product API** :
    1. Nombre d'appels **GET /api/products**
    2. Nombre d'appels **POST /api/products**

3. **Docker** :
   - Déploiement des conteneurs Prometheus, Grafana et Loki sur la VM monitoring via Docker Compose.

Livrables attendus Monitoring

1. **Fichiers de configuration** :
   - `prometheus.yml` et `docker-compose.yml`.
2. **Captures d'écran** :
   - Conteneurs Docker actifs (`docker ps`)
   - Dashboards Grafana GET et POST
   - Interface Prometheus
   - Logs visibles dans Grafana
3. **Documentation** :
   - Explication de la configuration Prometheus, Grafana et Loki.
   - URL pour accéder aux dashboards.

Critères d'évaluation Monitoring (6 points)

Critère	Points	Notes
Spring Boot configuré avec endpoint Prometheus et logs accessibles	1	`/actuator/prometheus` et logs envoyés vers Loki
Prometheus configuré correctement (scrape interval 10s,	1	Scraping fonctionnel toutes les 10 secondes

Critère	Points	Notes
target)		
Grafana configuré avec Prometheus et Loki comme datasources	1	Sources opérationnelles
Dashboard GET `/api/tasks` (Task API) ou `/api/products` (Product API) créé et métrique affichée	1	Affichage du nombre d'appels GET
Dashboard POST `/api/tasks` (Task API) ou `/api/products` (Product API) créé et métrique affichée	1	Affichage du nombre d'appels POST
Logs et traçabilité visibles dans Grafana	1	Logs centralisés et consultables

Total : 6 points

Partie 3️⃣ : GitLab CI/CD – Pipeline Docker

Objectif

Créer un pipeline GitLab CI/CD pour automatiser **tests, build et déploiement Docker** de l'application.

Consignes GitLab CI

1. **Pipeline GitLab CI** :
   - Fichier `.gitlab-ci.yml` à la racine.
   - 3 jobs : `test` (tests unitaires), `build` (construction du JAR), `docker` (build et push Docker).
2. **Variables GitLab** :
   - `DOCKER_USERNAME` → username Docker Hub
   - `DOCKER_PASSWORD` → mot de passe Docker Hub
   - `DOCKER_IMAGE` → nom de l'image Docker
3. **Nom des jobs** : `test`, `build`, `docker`
4. **Pipeline déclenché à chaque push sur** `develop`

Livrables attendus GitLab CI

1. **Fichier** `.gitlab-ci.yml` à la racine
2. **Captures d'écran / logs** :
   - Pipeline GitLab s'exécutant avec succès
   - Job `test` : tests unitaires passés
   - Job `build` : JAR généré
   - Job `docker` : image Docker poussée sur Docker Hub
3. **Documentation** :
   - Explication du pipeline et des variables
   - Nom exact de l'image publiée sur Docker Hub

Critères d'évaluation GitLab CI (2 points)

Critère	Points	Notes
`.gitlab-ci.yml` présent et jobs configurés correctement	1	Fichier à la racine, syntaxe correcte, jobs nommés correctement
Jobs test, build et docker exécutés avec succès	1	Tests passent, JAR créé, image poussée sur Docker Hub

Total : 2 points

Les 2 points restants de la note globale sont attribués à la qualité de la vidéo et du lien Notion :

Qualité Notion (1 point)

Critère	Points	Notes
Structure claire et lisible (titres, sections par partie)	0.25	Une section par partie : Kubernetes, Monitoring, CI/CD
Captures d'écran présentes et lisibles pour chaque partie	0.50	Toutes les captures demandées dans chaque section sont présentes et suffisamment zoomées pour être lisibles
Contenu des fichiers de config inclus (`mysql.yaml`, `l3gl-`	0.25	Fichiers copiés en blocs de code dans Notion, pas juste un lien

Critère	Points	Notes
`devops.yaml`, `prometheus.yml`, `docker-compose.yml`, `.gitlab-ci.yml`)		

Qualité Vidéo (1 point)

Critère	Points	Notes
Durée comprise entre 7 et 15 minutes	0.25	Toute vidéo hors de cette plage ne pourra pas obtenir ces points
Les 3 parties sont démontrées dans l'ordre (K8s → Monitoring → CI/CD)	0.25	Chaque partie visible à l'écran, pas juste mentionnée à l'oral
L'application est testée en direct (requêtes API visibles)	0.25	Au moins un appel GET et un appel POST montrés dans la vidéo
Son audible et explication orale présente	0.25	Les étudiants commentent ce qu'ils font, pas une vidéo muette

---

Date de livraison : Mercredi 08 Avril à 23h59